

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/sentiment_project/1.
                  encoding='latin-1',
                  header=None,
                  names=['target', 'id', 'date', 'flag', 'user', 'text'])

print("Done loading!")
```

Done loading!

```
df.head(10)
```

	target	id	date	flag	user	text
0	0	1467810369	Mon Apr 06 22:19:45 PDT 2009	NO_QUERY	_TheSpecialOne_	@switchfoot http://twitpic.com/2y1zl - Awww, t...
1	0	1467810672	Mon Apr 06 22:19:49 PDT 2009	NO_QUERY	scotthamilton	is upset that he can't update his Facebook by ...
2	0	1467810917	Mon Apr 06 22:19:53 PDT 2009	NO_QUERY	mattycus	@Kenichan I dived many times for the ball. Man...
3	0	1467811184	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	ElleCTF	my whole body feels itchy and like its on fire
4	0	1467811193	Mon Apr 06 22:19:57 PDT 2009	NO_QUERY	Karoli	@nationwideclass no, it's not behaving at all....
5	0	1467811372	Mon Apr 06 22:20:00 PDT 2009	NO_QUERY	joy_wolf	@Kwesidei not the whole crew
6	0	1467811551	Mon Apr 06 22:20:03 PDT 2009	NO_QUERY	...	...

```
print(df.shape)
```

(1600000, 6)

```
print(df['target'].value_counts())
```

```
target
0    800000
4    800000
Name: count, dtype: int64
```

```
df = df.sample(n=20000, random_state=42)
print(df.shape)
```

(20000, 6)

```
print(df['target'].value_counts())
```

```
target
4    10058
```

```
0      9942
Name: count, dtype: int64
```

```
df = df[['text', 'target']]
df.head(5)
```

	text	target
541200	@chrishasboobs AHHH I HOPE YOUR OK!!!	0
750	@misstoriblack cool , i have no tweet apps fo...	0
766711	@TiannaChaos i know just family drama. its la...	0
285055	School email won't open and I have geography ...	0
705995	upper airways problem	0

```
df['target'] = df['target'].replace(4, 1)
print(df['target'].value_counts())
```

```
target
1      10058
0       9942
Name: count, dtype: int64
```

```
import re
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words('english'))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
def clean_tweet(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'^a-z\s', '', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return ' '.join(words)
```

```
df['clean_text'] = df['text'].apply(clean_tweet)
print("Cleaning done!")
```

```
Cleaning done!
```

```
df[['text', 'clean_text']].head(10)
```

	text	clean_text
541200	@chrisasboobs AHHH I HOPE YOUR OK!!!	ahhh hope ok
750	@misstoriblack cool , i have no tweet apps fo...	cool tweet apps razr
766711	@TiannaChaos i know just family drama. its la... know family drama lamehey next time u hang kim...	
285055	School email won't open and I have geography ... school email wont open geography stuff revise ...	
705995	upper airways problem	upper airways problem
379611	Going to miss Pastor's sermon on Faith...	going miss pastors sermon faith
1189018	on lunch....dj should come eat with me	lunchdj come eat
667030	@piginthepoke oh why are you feeling like that?	oh feeling like
93541	gahh noo!peyton needs to live!this is horrible	gahh noopeyton needs livethis horrible
1097326	@mrstessyman thank you glad you like it! There... thank glad like product review bit site enjoy ...	

```
df[['clean_text', 'target']].head(10)
```

	clean_text	target
541200	ahhh hope ok	0
750	cool tweet apps razr	0
766711	know family drama lamehey next time u hang kim...	0
285055	school email wont open geography stuff revise ...	0
705995	upper airways problem	0
379611	going miss pastors sermon faith	0
1189018	lunchdj come eat	1
667030	oh feeling like	0
93541	gahh noopeyton needs livethis horrible	0
1097326	thank glad like product review bit site enjoy ...	1

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df['clean_text'])
y = df['target']
```

```
print(X.shape)
print(y.shape)
```

```
(20000, 5000)
(20000,)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, te

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

Training size: (16000, 5000)
 Testing size: (4000, 5000)

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
import numpy as np

# model
log_reg_model = LogisticRegression(max_iter=1000)
print("Model initialized")

# K-Fold cross validation (k=5)
log_reg_scores = cross_val_score(log_reg_model, X, y, cv=5,

print("Logistic Regression K-Fold Scores:", log_reg_scores)
print("Average Accuracy:", round(np.mean(log_reg_scores) * 1
print("Standard Deviation:", round(np.std(log_reg_scores) *
```

Model trained
 Logistic Regression K-Fold Scores: [0.73875 0.73375 0.74125 0.7335 0.7345]
 Average Accuracy: 73.64 %
 Standard Deviation: 0.31 %

```
from sklearn.metrics import classification_report

# Final fit on training data
log_reg_model.fit(X_train, y_train)

# Predict on test data
y_pred_log_reg = log_reg_model.predict(X_test)

# Classification report
print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_log_reg))
```

Logistic Regression Classification Report:

	precision	recall	f1-score	support
0	0.74	0.70	0.72	1987
1	0.72	0.75	0.74	2013
accuracy			0.73	4000
macro avg	0.73	0.73	0.73	4000
weighted avg	0.73	0.73	0.73	4000

```
from sklearn.naive_bayes import MultinomialNB

# Train model
bayes_model = MultinomialNB()
```

```

print("Model initialized")

# K-Fold cross validation (k=5)
bayes_scores = cross_val_score(bayes_model, X, y, cv=5, scoring='accuracy')

print("Naive Bayes K-Fold Scores:", bayes_scores)
print("Average Accuracy:", round(np.mean(bayes_scores) * 100, 2))
print("Standard Deviation:", round(np.std(bayes_scores) * 100, 2))

```

```

Model trained
Naive Bayes K-Fold Scores: [0.73175 0.72125 0.72825 0.72375 0.72575]
Average Accuracy: 72.61 %
Standard Deviation: 0.36 %

```

```

# Final fit on training data
bayes_model.fit(X_train, y_train)

# Predict on test data
y_pred_bayes = bayes_model.predict(X_test)

# Classification report
print("Naive Bayes Classification Report:")
print(classification_report(y_test, y_pred_bayes))

```

```

Naive Bayes Classification Report:

```

	precision	recall	f1-score	support
0	0.73	0.72	0.73	1987
1	0.73	0.73	0.73	2013
accuracy			0.73	4000
macro avg	0.73	0.73	0.73	4000
weighted avg	0.73	0.73	0.73	4000

```

import tensorflow as tf
from tensorflow import keras
from keras import layers

# Build the model (build just the architecture)
ann_model = keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(5000,)),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

# Compile the model (just setup the rules) (no training has
ann_model.compile(optimizer='adam',

```

```
loss='binary_crossentropy',
metrics=['accuracy'])
```

Summary

```
ann_model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape` to `Dense` layer. It is inferred from the input data.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 256)	1,280,256
dropout_4 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 128)	32,896
dropout_5 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 1)	129

Total params: 1,313,281 (5.01 MB)

Trainable params: 1,313,281 (5.01 MB)

Non-trainable params: 0 (0.00 B)

```
from sklearn.model_selection import StratifiedKFold
import numpy as np
```

Convert sparse matrix to dense for Keras

```
X_dense = X.toarray()
```

K-Fold setup

```
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
ann_scores = []
```

```
fold = 1
```

```
for train_idx, test_idx in kfold.split(X_dense, y):
    print(f"Training Fold {fold}...")
```

Split data

```
X_fold_train, X_fold_test = X_dense[train_idx], X_dense[test_idx]
y_fold_train, y_fold_test = y.iloc[train_idx], y.iloc[test_idx]
```

Build fresh model each fold

```
model_fold = keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(50,)),
    layers.Dropout(0.3),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
```

```

])

model_fold.compile(optimizer='adam',
                   loss='binary_crossentropy',
                   metrics=['accuracy'])

# Train
model_fold.fit(X_fold_train, y_fold_train,
               epochs=5,
               batch_size=32,
               verbose=0)

# Evaluate
_, accuracy = model_fold.evaluate(X_fold_test, y_fold_test)
ann_scores.append(accuracy)
print(f"Fold {fold} Accuracy: {round(accuracy * 100, 2)}")
fold += 1

print("\nANN K-Fold Scores:", [round(s * 100, 2) for s in ann_scores])
print("Average Accuracy:", round(np.mean(ann_scores) * 100, 2))
print("Standard Deviation:", round(np.std(ann_scores) * 100, 2))

```

```

Training Fold 1...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `Input` layers.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Fold 1 Accuracy: 70.92%
Training Fold 2...
Fold 2 Accuracy: 70.25%
Training Fold 3...
Fold 3 Accuracy: 70.0%
Training Fold 4...
Fold 4 Accuracy: 70.4%
Training Fold 5...
Fold 5 Accuracy: 70.82%

ANN K-Fold Scores: [70.92, 70.25, 70.0, 70.4, 70.82]
Average Accuracy: 70.48 %
Standard Deviation: 0.35 %

```

```

from sklearn.metrics import classification_report

# Final fit on full training data
ann_model.fit(X_fold_train, y_fold_train,
              epochs=5,
              batch_size=32,
              verbose=1)

# Predict on test data
y_pred_ann_prob = ann_model.predict(X_dense[test_idx])
y_pred_ann = (y_pred_ann_prob > 0.5).astype(int)

```

```
# Classification report
print("ANN Classification Report:")
print(classification_report(y_fold_test, y_pred_ann))
```

```
Epoch 1/5
500/500 ————— 10s 21ms/step - accuracy: 0.9795 - loss: 0.0568
Epoch 2/5
500/500 ————— 10s 20ms/step - accuracy: 0.9814 - loss: 0.0472
Epoch 3/5
500/500 ————— 9s 18ms/step - accuracy: 0.9838 - loss: 0.0370
Epoch 4/5
500/500 ————— 9s 18ms/step - accuracy: 0.9858 - loss: 0.0335
Epoch 5/5
500/500 ————— 10s 20ms/step - accuracy: 0.9875 - loss: 0.0275
125/125 ————— 1s 4ms/step
```

```
ANN Classification Report:
              precision    recall  f1-score   support

     0       0.69       0.73       0.71       1988
     1       0.72       0.68       0.70       2012

 accuracy         0.70
 macro avg       0.70       0.70       0.70       4000
weighted avg       0.71       0.70       0.70       4000
```

MODEL COMPARISON:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Summary table
results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Naive Bayes', 'ANN'],
    'K-Fold Accuracy (%)': [
        round(np.mean(log_reg_scores) * 100, 2),
        round(np.mean(bayes_scores) * 100, 2),
        round(np.mean(ann_scores) * 100, 2)
    ],
    'Std Deviation (%)': [
        round(np.std(log_reg_scores) * 100, 2),
        round(np.std(bayes_scores) * 100, 2),
        round(np.std(ann_scores) * 100, 2)
    ]
})

print("=" * 45)
print("          MODEL COMPARISON SUMMARY")
print("=" * 45)
print(results.to_string(index=False))
```

```
print("=" * 45)
print(f"🏆 Best Model: {results.loc[results['K-Fold Accuracy (%)'] == 73.64].Model[0]}
```

```
=====
MODEL COMPARISON SUMMARY
=====
      Model  K-Fold Accuracy (%)  Std Deviation (%)
Logistic Regression           73.64             0.31
      Naive Bayes           72.61             0.36
              ANN           70.48             0.35
=====
🏆 Best Model: Logistic Regression
```

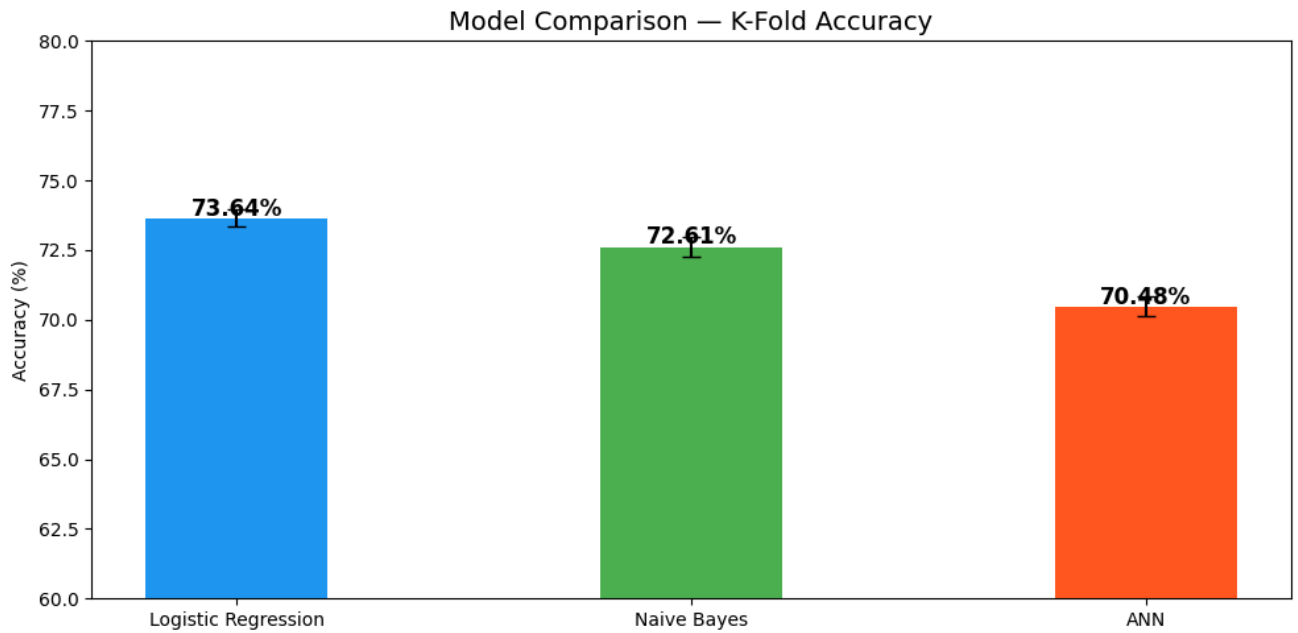
```
# Bar chart comparison
plt.figure(figsize=(10, 5))
colors = ['#2196F3', '#4CAF50', '#FF5722']

bars = plt.bar(results['Model'],
                results['K-Fold Accuracy (%)'],
                color=colors,
                width=0.4)

# Add value labels on top of each bar
for bar, val in zip(bars, results['K-Fold Accuracy (%)']):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.1,
             f'{val}%',
             ha='center',
             fontsize=12,
             fontweight='bold')

# Add error bars for std deviation
plt.errorbar(results['Model'],
             results['K-Fold Accuracy (%)'],
             yerr=results['Std Deviation (%)'],
             fmt='none',
             color='black',
             capsize=5)

plt.title('Model Comparison – K-Fold Accuracy', fontsize=14)
plt.ylabel('Accuracy (%)')
plt.ylim(60, 80)
plt.tight_layout()
plt.show()
```

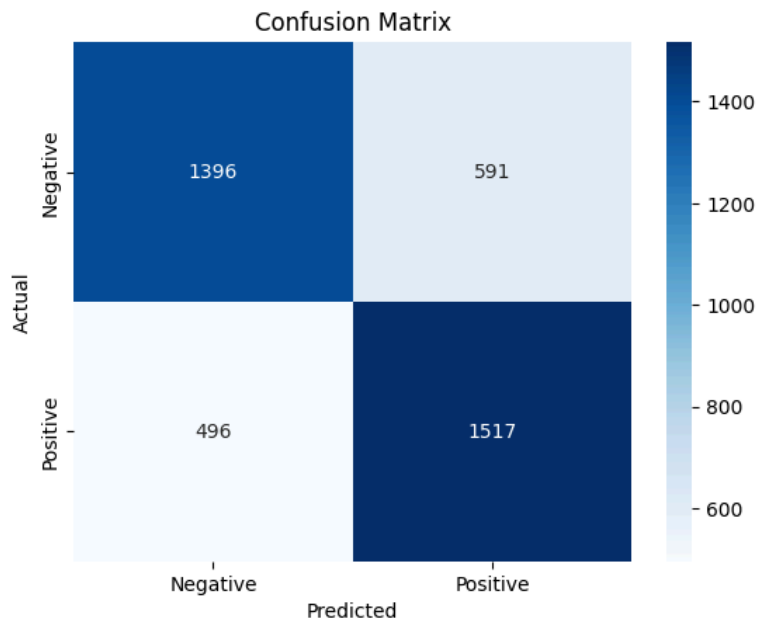


```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_log_reg)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])

plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



```
from wordcloud import WordCloud
```

```
# Separate positive and negative tweets
```

```
positive_tweets = ' '.join(df[df['target'] == 1]['clean_text'])
```

```
negative_tweets = ' '.join(df[df['target'] == 0]['clean_text'])
```

```
# Plot them side by side
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Positive word cloud
```

```
wc_pos = WordCloud(width=600, height=400, background_color='white')
```

```
axes[0].imshow(wc_pos)
```

```
axes[0].set_title('Positive Tweets')
```

```
axes[0].axis('off')
```

```
# Negative word cloud
```

```
wc_neg = WordCloud(width=600, height=400, background_color='white')
```

```
axes[1].imshow(wc_neg)
```

```
axes[1].set_title('Negative Tweets')
```

```
axes[1].axis('off')
```

```
plt.show()
```

